NYU

ROB-UY 3303

Robot Motion & Planning

Lecture 08A - Motion Planning

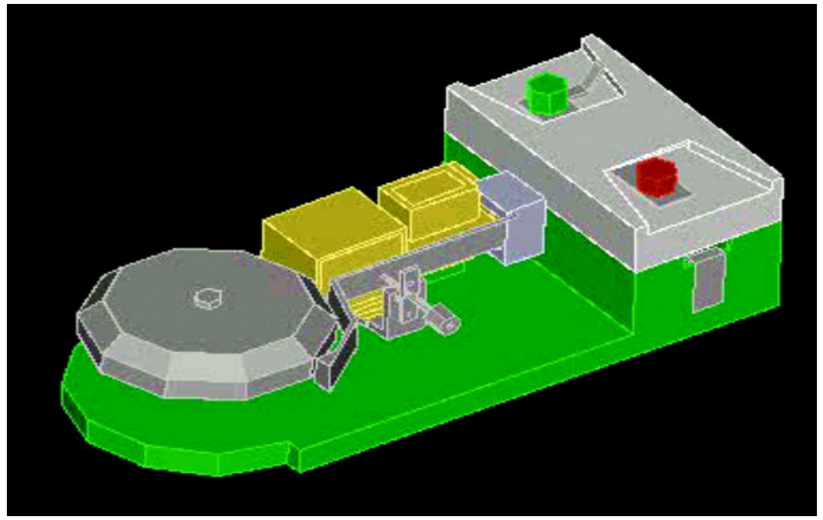


Agenda

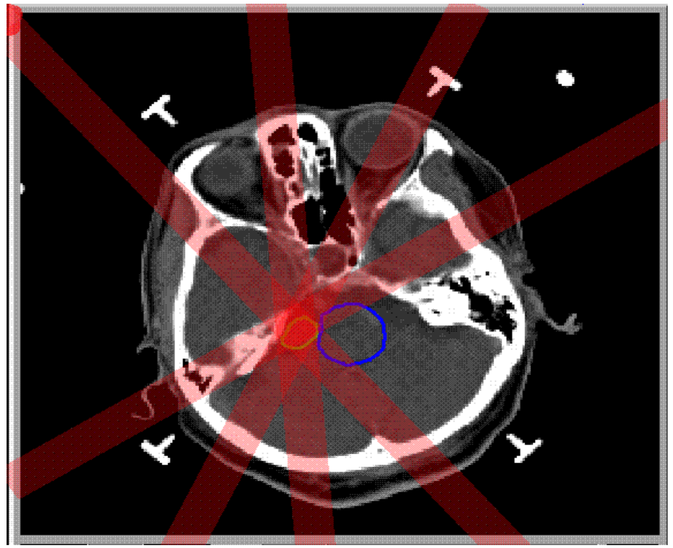
1. Degrees of Freedom
2. **Configuration Space for Planning**
3. Coordinate Frames


SPARSE-RRT
A Kinodynamic Motion Planning Solution
with Quality Assurance

- Applications



Assembly Planning, Latombe

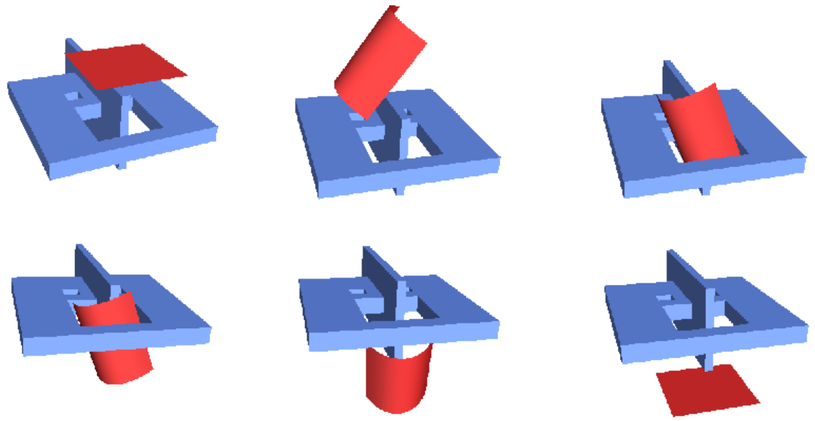


Cross-Firing of a Tumor, Latombe

- Applications



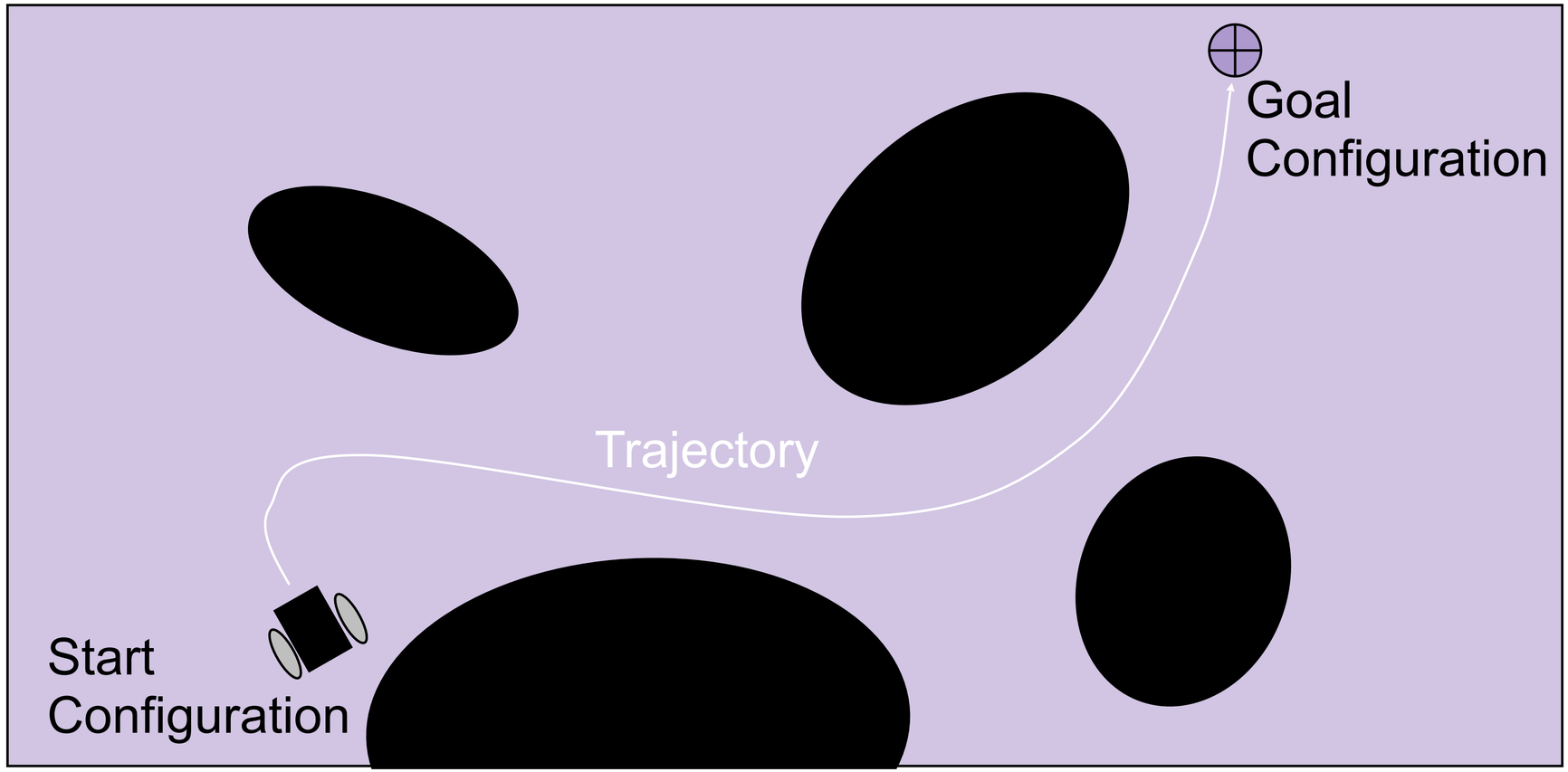
Tomb Raider 3 (Eidos Interactive)



Deformable Objects, Kavraki

- **The problem:**

  - To construct a collision-free path from some initial configuration to some goal configuration for a robot within a workspace containing obstacles.


Goal
Configuration
Trajectory
Start
Configuration

- **Inputs**
  - Geometry of robots and world
  - Kinematics/Dynamics of robots and world
  - Start and Goal configurations

- **Outputs**
  - Continuous sequence of configurations connecting the start and goal configurations

- **Extensions**
  - Moving obstacles
  - Multiple robots
  - Movable objects
  - Assembly planning
  - Goal is to acquire information by sensing
  - Nonholonomic constraints
  - Dynamic constraints
  - Stability constraints
  - Uncertainty in model, control and sensing
  - Exploiting task mechanics (under-actuated systems)
  - Physical models and deformable objects
  - Integration with higher-level planning

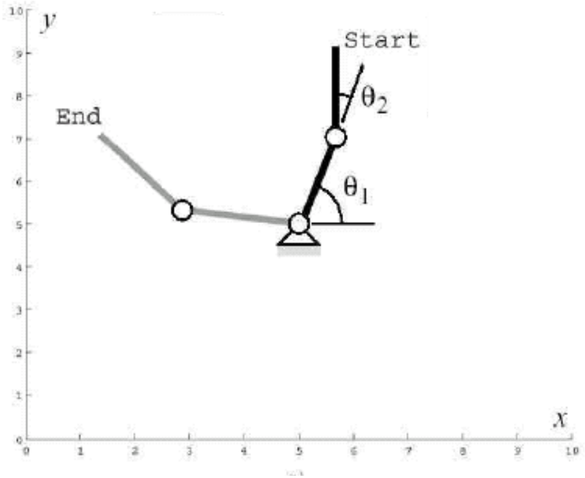
- **The Configuration Space**

  - To facilitate motion planning, the **configuration space** was defined as a tool that can be used with planning algorithms.

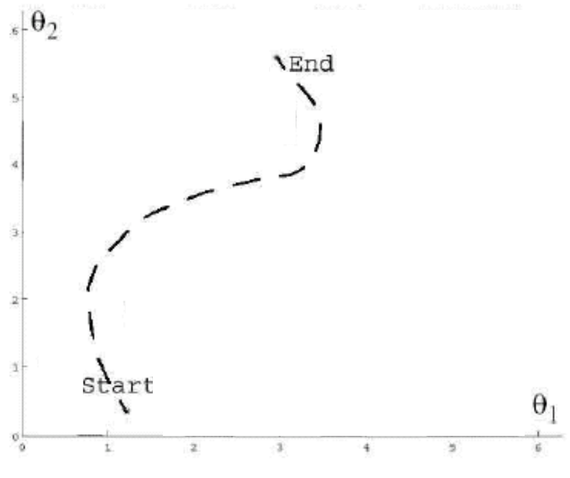
JC Latombe (1991)

Configuration Space For Planning

- A **configuration** q will completely define the position of every point on a robot
- The **configuration space** C, is the space of all possible configurations of the robot.
- The **free space** $F \in C$, is the portion of the configuration space which is collision-free.



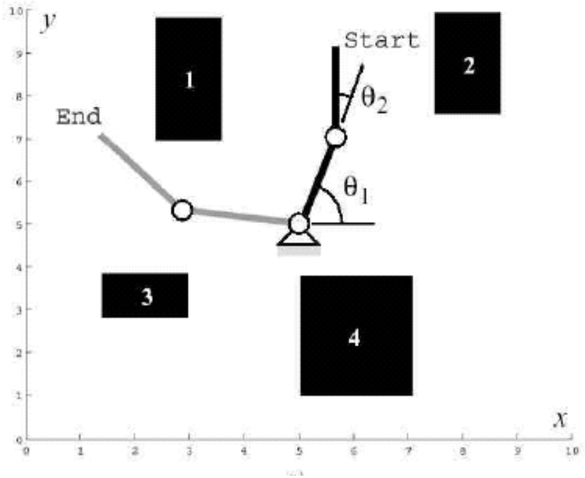
Workspace



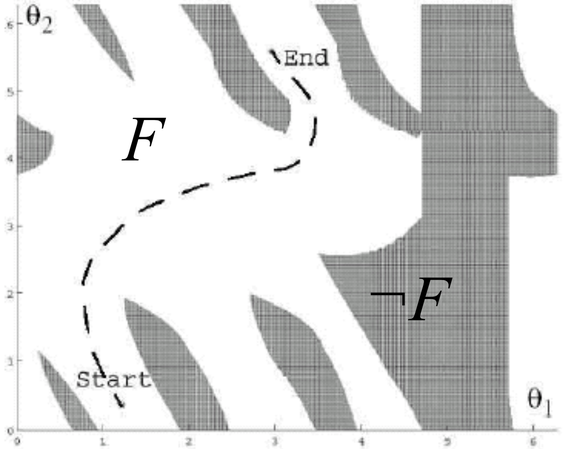
Configuration Space

Configuration Space For Planning

- A **configuration** q will completely define the state of a robot (e.g. mobile robot x, y, θ)
- The **configuration space** C, is the space of all possible configurations of the robot.
- The **free space** $F \in C$, is the portion of the free space which is collision-free.



Workspace



Configuration Space

- **The Configuration Space**

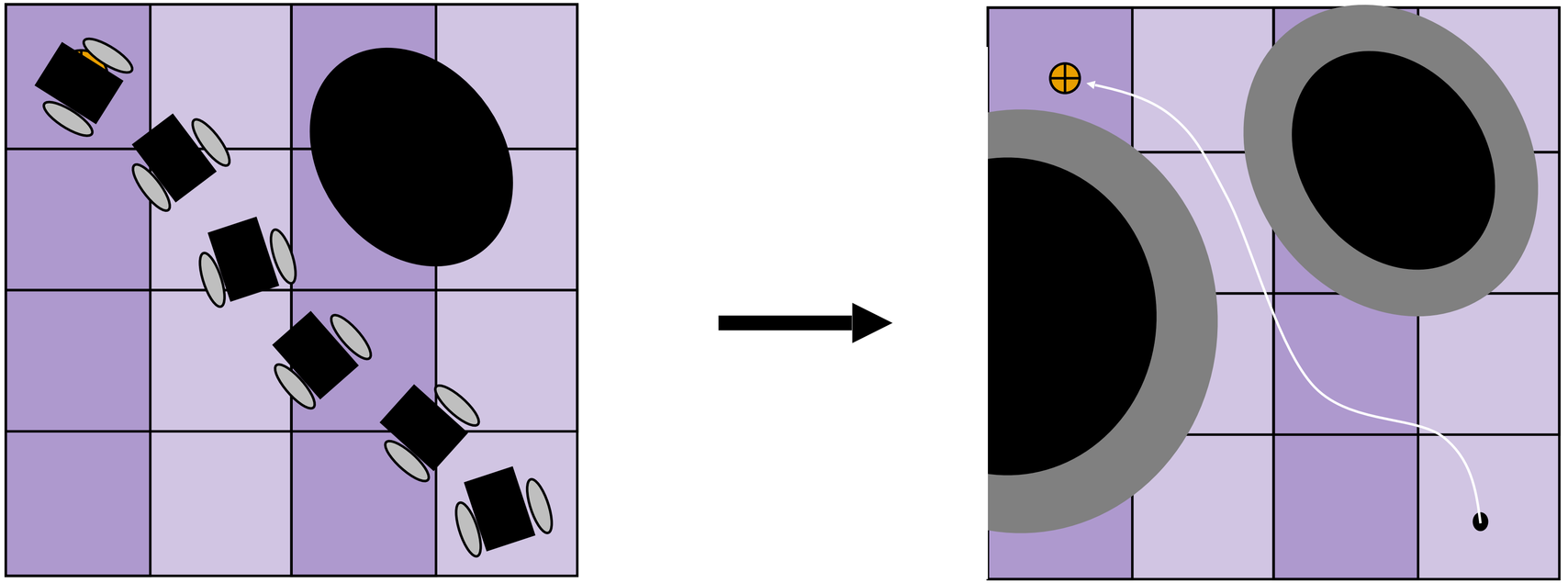
  - The goal of motion planning then, is to find a path in F that connects the initial configuration q_{start} to the goal configuration q_{goal}

- **General Approach to MP**

1. (Re)define C
2. Discretize C
3. Search the Discretization of C

- **Redefining C for each different MPP**
  - Example 1: Include 3DOF for a mobile robot in static environment - (x,y,θ).
  - Example 2: Include only 2DOF for a mobile robot in static environment - (x,y).
  - Example 3: Include 5DOF for a mobile robot in dynamic environment - (x,y,θ,v,t).

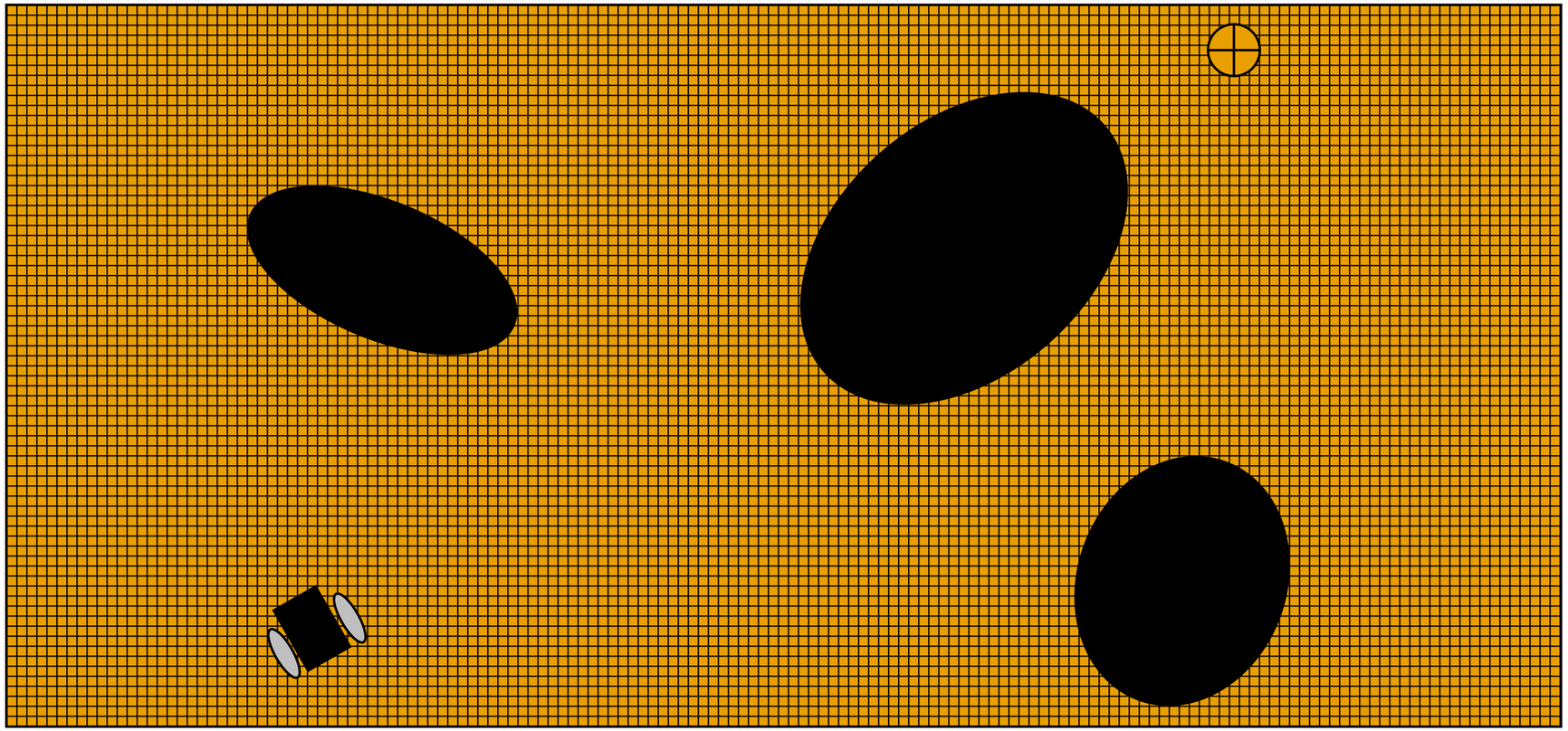
- **Redefining C: Plan paths for a point robot**
  - Instead of using a robot of fixed dimensions/size, “grow” the obstacles to reflect how close the robot can get.



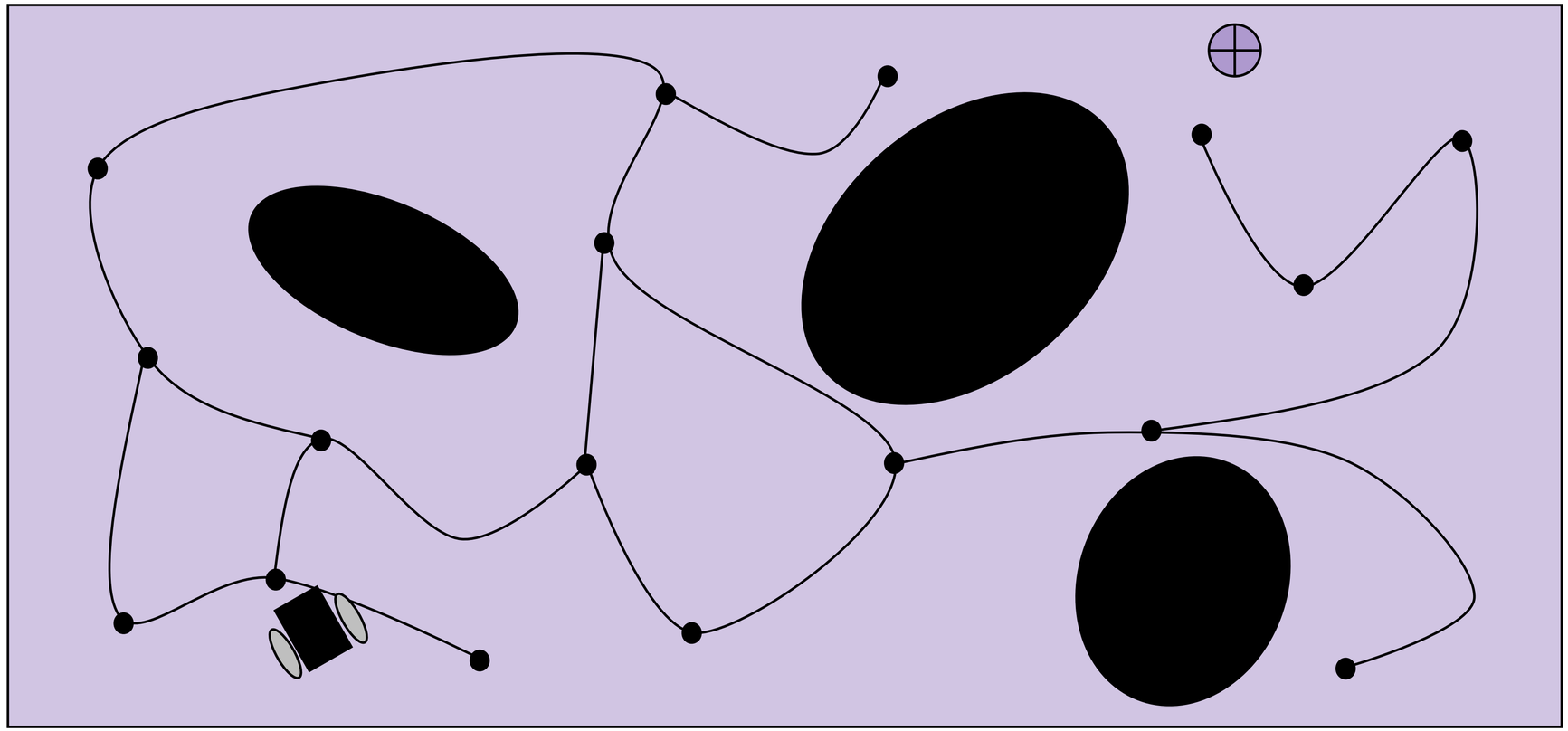
- **Discretizing C for each different MPP**

  - Example 1: Cell decomposition
  - Example 2: Roadmap
  - Example 3: Potential field

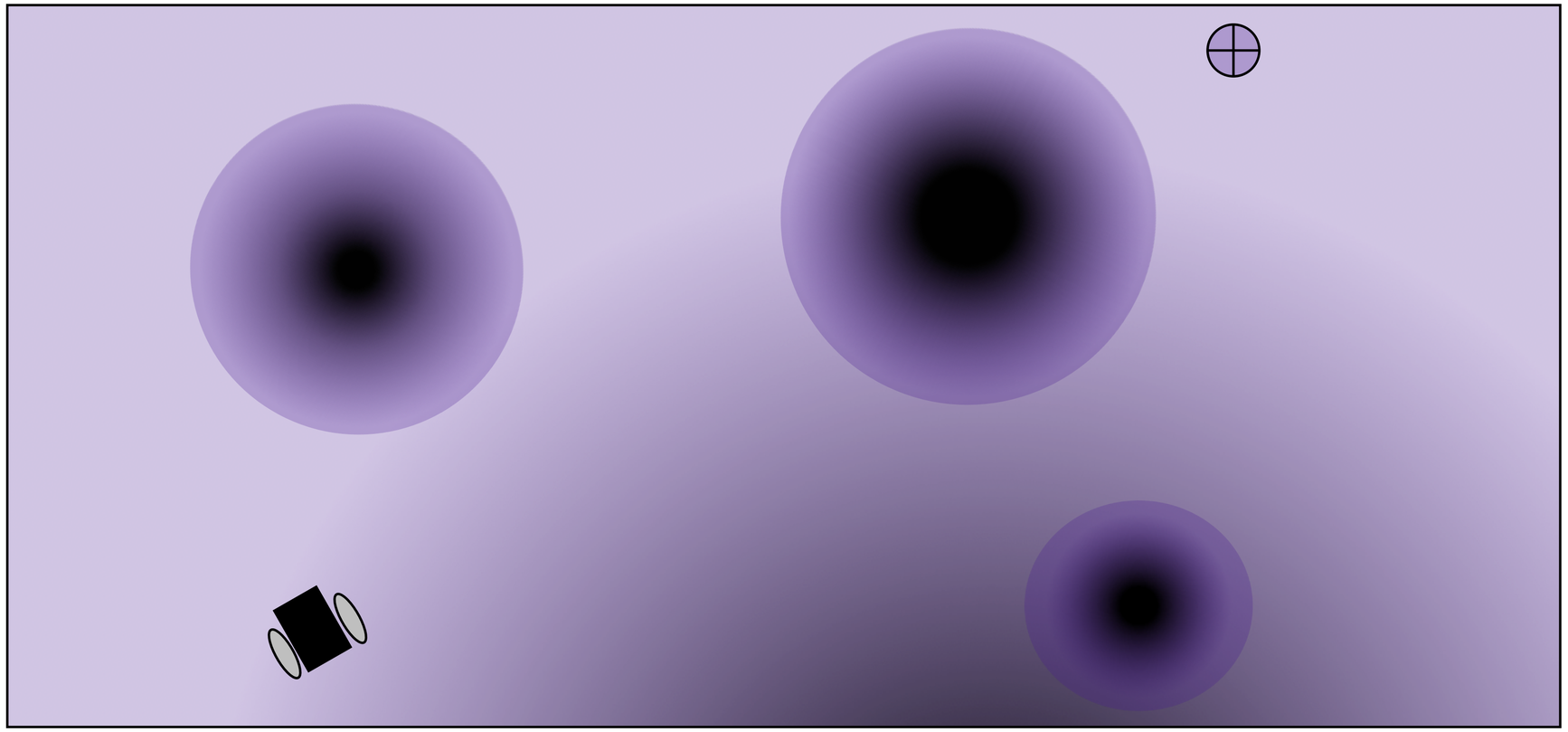
- **Discretizing C for each different MPP**
  - Example 1: Cell decomposition



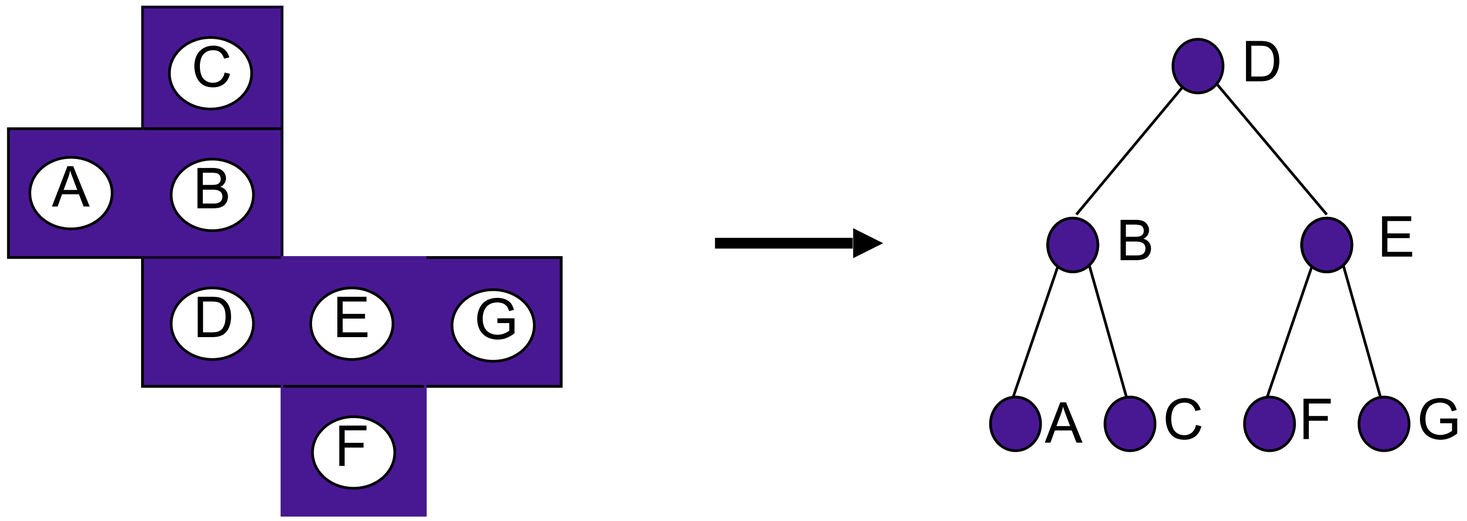
- **Discretizing C for each different MPP**
  - Example 2: Roadmap



- **Discretizing C for each different MPP**
  - Example 3: Potential field



- **Searching C for a trajectory**
  - E.g. in a map of cells, find a route from D to F



- **Metrics**
  - How we compare MP algorithms and trajectories:

1. Speed or Complexity
2. Completeness
3. Optimality
4. Feasibility of solutions

Metrics – Speed or Runtime Complexity

- Often, planners are compared based on the **running time** of an algorithm.
  - Must specify the hardware when reporting, (e.g. processor type, …)

- Example:
  - Planner A outperformed Planner B in that it took half the time to solve the same planning problem.

Metrics – Speed or Runtime Complexity

- Planners are also compared based on the algorithm's **run time complexity**
  - i.e. the number of steps or operations an algorithm must take as a function of the size of the input.

- **Metrics – Speed or Runtime Complexity**
  - Example: For M particles and N sensors, calculate the weights assuming expected measurements are known

```
for (int i=0; i<M; i++) {
     w(i) = 0.0001;
     for (int j=0; j<N; j++){
          w(i) *= gauss(z-z_exp(i,j));
     }
}
```

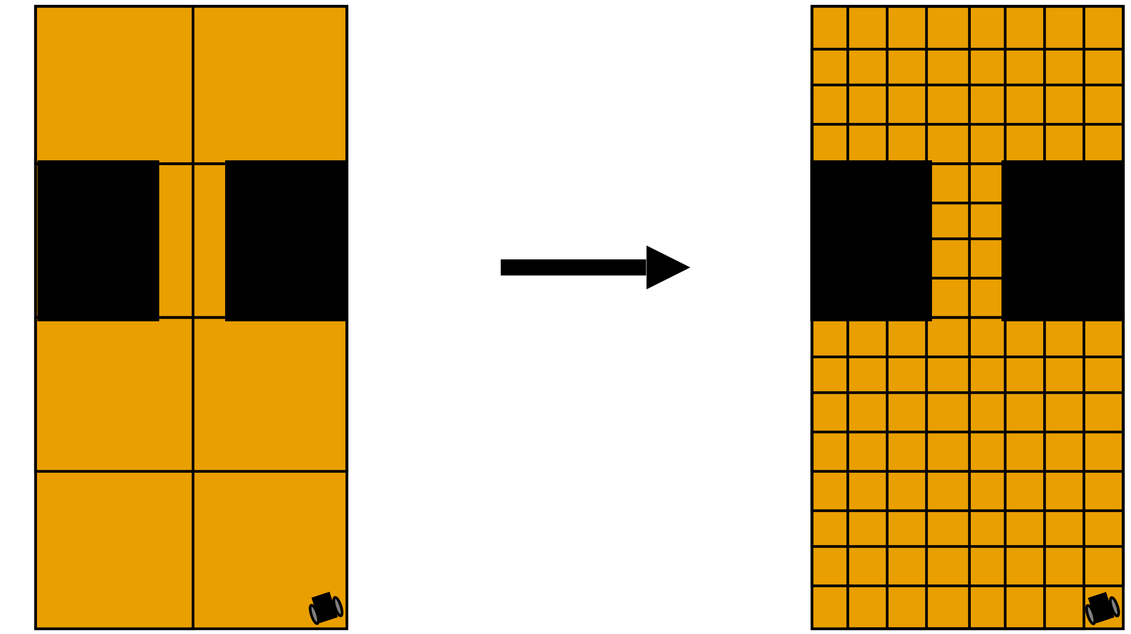
  - In this example there are on the order of MxN operations, i.e. $O(MN)$

- **Metrics - Completeness**
  - A **complete** algorithm is one that is guaranteed to find a solution if one exists, or determine if no solution exists.

  - Time Consuming!
    - An exhaustive search will search every possible path to see if it is a feasible solution.
    - A complete planner usually requires exponential time in the number of degrees of freedom, objects, etc.

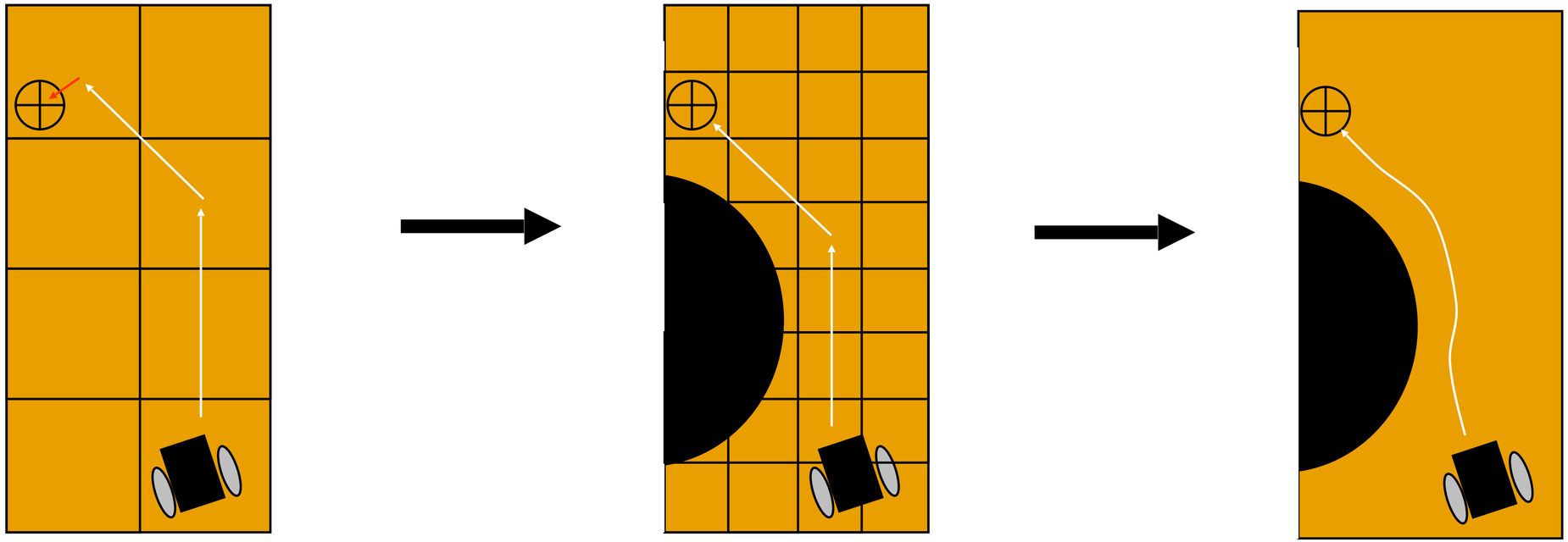
Metrics - Completeness

- A **resolution complete** planner discretizes the space and returns a path whenever one exists in the discretized representation.



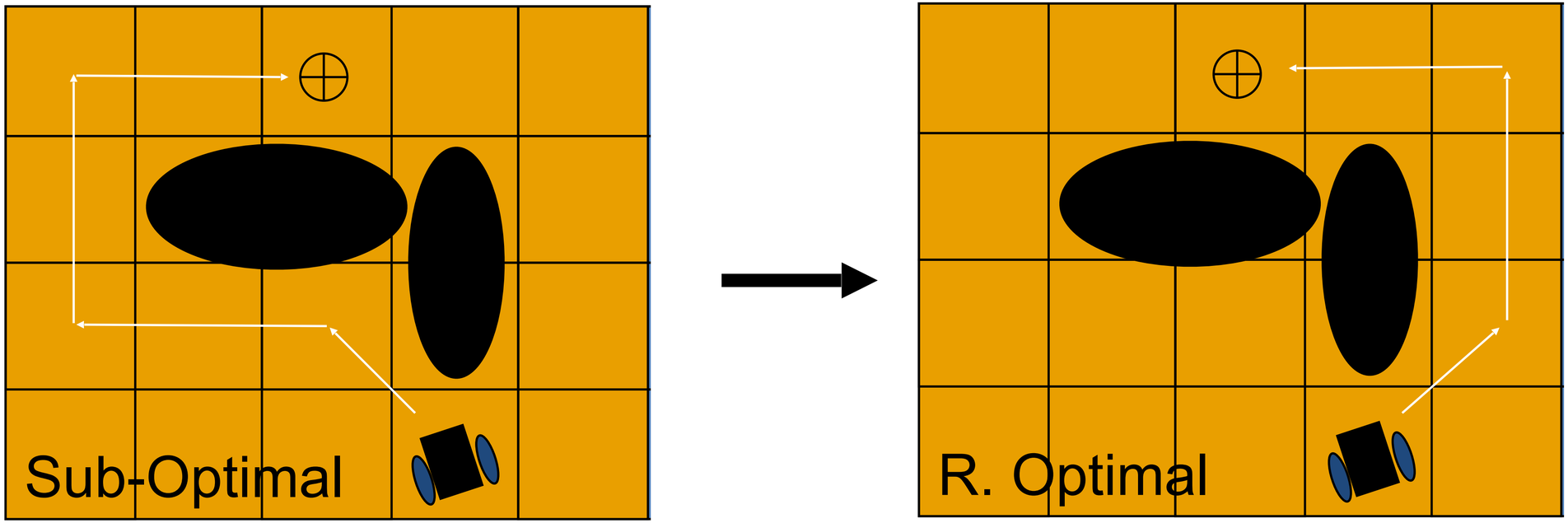
- **Metrics - Completeness**
  - A **probabilistically complete** planner returns a path with high probability if a path exists. It may not terminate if no path exists.
    - E.g. $P(failure) \rightarrow 0$ as $\text{time} \rightarrow \infty$
  - Weaker form of completeness, but usually faster.

- **Metrics - Optimality**
  - E.g. shortest path, minimum time, lowest energy
  - Resolution of Discretization can lead to sub-optimal solutions

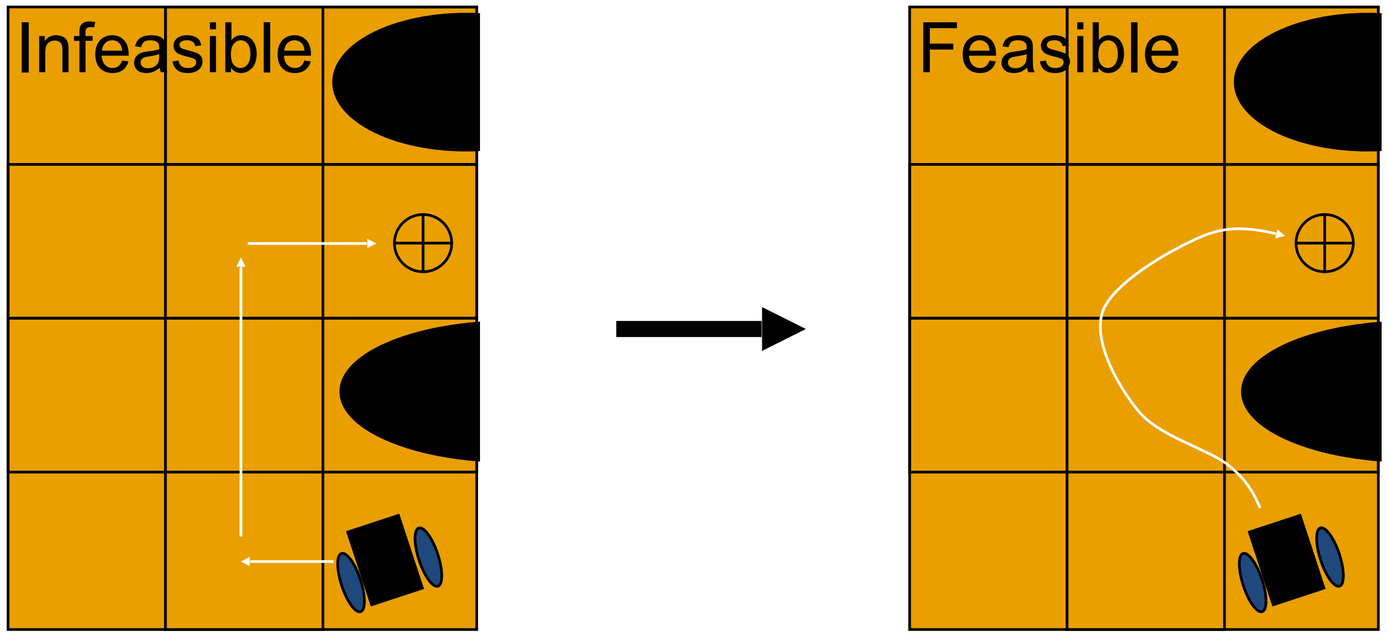


Metrics - Optimality

- Some algorithms will only guarantee sub-optimal solutions (e.g. Greedy Search).



- **Metrics - Feasibility**
  - Not all planners consider the exact model of the robot.



- **Metrics - Overview**
  - Theoretical algorithms
    - Strive for completeness and minimal worst-case complexity
    - Difficult to implement
  - Heuristic algorithms
    - Strive for efficiency in common situations
    - Use simplifying assumptions
    - Weaker completeness
    - Exponential algorithms that work in practice

- **Metrics**
  1. Speed or Complexity
  2. Completeness
  3. Optimality
  4. Feasibility of solutions

